

Импорты

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from scipy.stats import chi2_contingency
from itertools import combinations
import ipywidgets as widgets
from IPython.display import display
```

```
pd.set_option('display.float_format', '{:.2f}'.format)
np.set_printoptions(suppress=True, precision=4)
```

```
df = pd.read_excel("/content/data_1_2_well_cleanedv2.xlsx")
df.head(3)
```

Добавление новых переменных

```
max_hours = df['avr_work_hours'].max()
bins = list(range(0, int(max_hours) + 10, 10))

labels = [f"{bins[i]}-{bins[i+1]}" for i in range(len(bins)-1)]

df['avr_work_hours_grouped'] = pd.cut(
    df['avr_work_hours'],
    bins=bins,
    labels=labels,
    right=False
)

print(df[['avr_work_hours', 'avr_work_hours_grouped']].head())
```

	avr_work_hours	avr_work_hours_grouped
0	44.00	40-50
1	22.50	20-30
2	22.50	20-30
3	22.50	20-30
4	22.50	20-30

Функции

```
def plot_burnout_by_features(features, target_col='burnout1_desc', desired_order=None, colors=None):

    dataframe = df.copy()
    if desired_order is None:
        if target_col == 'burnout1_desc':
            desired_order = ['Нет риска', 'Риск выгорания', 'Очень высокий риск выгорания']
            if colors is None:
                colors = ['#2d5a3c', '#ff9933', '#800000']
        else:
            desired_order = sorted(dataframe[target_col].dropna().unique().tolist())

    if colors is None:
        cmap = plt.get_cmap('Set1')
        colors = [cmap(i) for i in range(len(desired_order))]

    for feature in features:
        if feature not in dataframe.columns:
            print(f"{feature} отсутствует в DataFrame. Пропускаем.")
            continue

        temp_df = dataframe[[feature, target_col]].copy().dropna()
        if temp_df.empty: continue

        if temp_df[feature].nunique() > 15:
            try:
                temp_df[feature] = pd.qcut(temp_df[feature], q=5, duplicates='drop').astype(str)
            except Exception:
                temp_df[feature] = temp_df[feature].astype(str)
```

```

counts = temp_df[feature].value_counts()
ct = pd.crosstab(temp_df[feature], temp_df[target_col], normalize='index') * 100

for cat in desired_order:
    if cat not in ct.columns:
        ct[cat] = 0

ct = ct[desired_order]
ct = ct.sort_index(ascending=True)

counts = counts.reindex(ct.index)

fig, ax = plt.subplots(figsize=(12, 8))
ct.plot(kind='bar', stacked=True, color=colors, ax=ax, width=0.8)

for p in ax.patches:
    h = p.get_height()
    if h > 3:
        ax.text(
            p.get_x() + p.get_width()/2.,
            p.get_y() + h/2.,
            f'{int(h)}%',
            ha='center', va='center',
            color='white', fontsize=9, fontweight='bold'
        )

for i, (idx, row) in enumerate(ct.iterrows()):
    n_total = counts[idx]
    ax.text(
        i, 102, f'N = {int(n_total)}',
        ha='center', va='bottom',
        fontsize=10, fontweight='bold', color='#333333'
    )

plt.title(f'Распределение в разрезе признака: {feature}', fontsize=16, pad=30)
plt.ylim(0, 115)

if target_col == 'burnout1_desc':
    plt.legend(title='Уровень риска выгорания', bbox_to_anchor=(1.02, 1), loc='upper left')
else:
    plt.legend(title=target_col, bbox_to_anchor=(1.02, 1), loc='upper left')

plt.ylabel('Процентное соотношение (%)')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

```

def trust_intervals(target, grouping_value = 'burnout1_desc', target_categories=['Очень высокий риск выгорания', 'Риск выг
for target_category in target_categories:
    alpha = 0.05
    contingency_table = pd.crosstab(df[target], df[grouping_value])

    groups = contingency_table.index.tolist()
    group_pairs = list(combinations(groups, 2))
    bonferroni_alpha = alpha / len(group_pairs)

    results = []

    for g1, g2 in group_pairs:
        count_target_g1 = contingency_table.loc[g1, target_category]
        count_others_g1 = contingency_table.loc[g1].sum() - count_target_g1

        count_target_g2 = contingency_table.loc[g2, target_category]
        count_others_g2 = contingency_table.loc[g2].sum() - count_target_g2

        table_2x2 = [
            [count_target_g1, count_others_g1],
            [count_target_g2, count_others_g2]
        ]

        try:
            chi2, p, dof, expected = chi2_contingency(table_2x2, correction=True)

            results.append({
                'Папа рпynn': f'{g1} vs {g2}',
                'p-value': round(float(p), 8),
                'Значимо (Бонферрони)': p < bonferroni_alpha
            })
        except ValueError:
            continue

results_df = pd.DataFrame(results)

```

```

significant_diffs = results_df[results_df['Значимо (Бонферрони)'] == True]

out_table, out_plot = widgets.Output(), widgets.Output()
with out_table:
    print(f"Анализ по категории: '{target_category}'")
    print(f"Порог Бонферрони (0.05 / {len(group_pairs)}): {bonferroni_alpha:.5f}")
    print(f"Найдено значимых различий: {len(significant_diffs)} из {len(group_pairs)}\n")
    display(significant_diffs.sort_values(by='p-value'))

total_counts = contingency_table.sum(axis=1)
target_counts = contingency_table[target_category]

p = target_counts / total_counts
se = np.sqrt(p * (1 - p) / total_counts)

ci = 1.96 * se

plot_df = pd.DataFrame({
    target: p.index,
    'proportion': p.values,
    'ci': ci.values
}).sort_values('proportion', ascending=False)

with out_plot:
    plt.figure(figsize=(8, 6))

    plt.errorbar(x=plot_df[target], y=plot_df['proportion'], yerr=plot_df['ci'],
                 fmt='o', color='crimson', ecolor='black', capsize=5,
                 markersize=7, elinewidth=2, label='95% Доверительный интервал')

    mean_proportion = target_counts.sum() / total_counts.sum()
    plt.axhline(y=mean_proportion, color='gray', linestyle='--', alpha=0.6, label='Среднее по выборке')

    plt.title(f'Отличия групп "{target_category}" по {target}', fontsize=14, pad=20)
    plt.ylabel('Доля (0.0 - 1.0)', fontsize=12)
    plt.xticks(rotation=45, ha='right')
    plt.grid(axis='y', linestyle=':', alpha=0.7)
    plt.legend()

    plt.tight_layout()
    plt.show()

print('\n')
display(widgets.HBox([out_table, out_plot]))

```

✎ Применение функций:

```

def get_differential_burnout_report(feature, target='burnout1_desc', target_categories=['Очень высокий риск выгорания', 'Р
plot_burnout_by_features([feature], target_col=target, desired_order=None, colors=None)
print()
trust_intervals(feature, grouping_value=target, target_categories=target_categories)

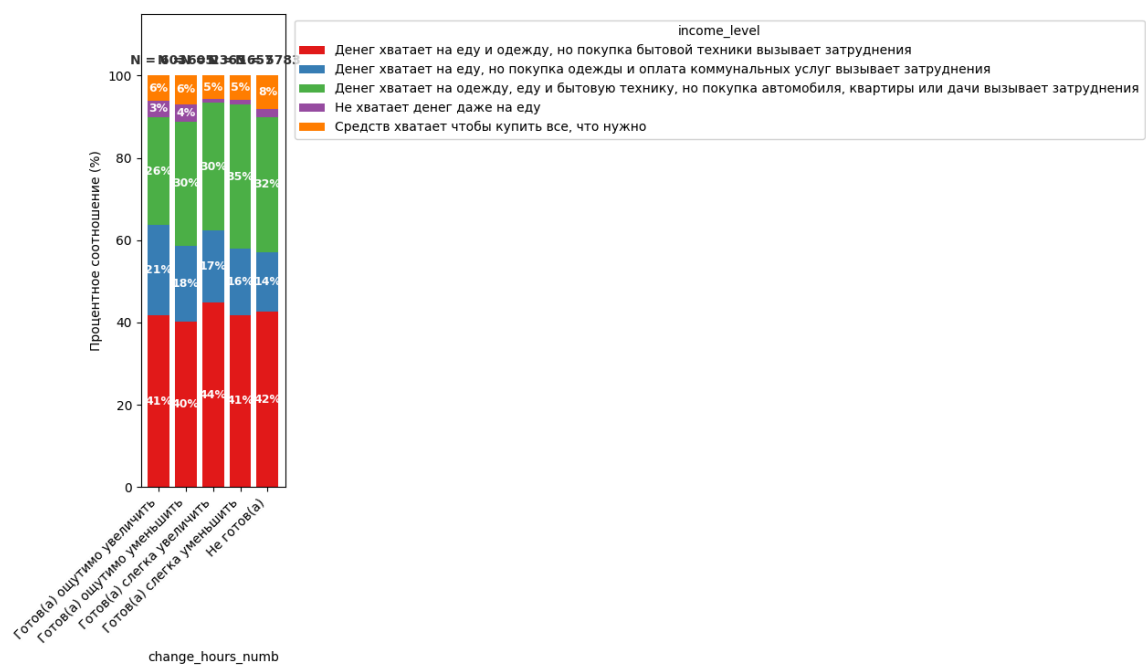
```

```

get_differential_burnout_report('change_hours_num', 'income_level', target_categories=['Средств хватает чтобы купить все,

```

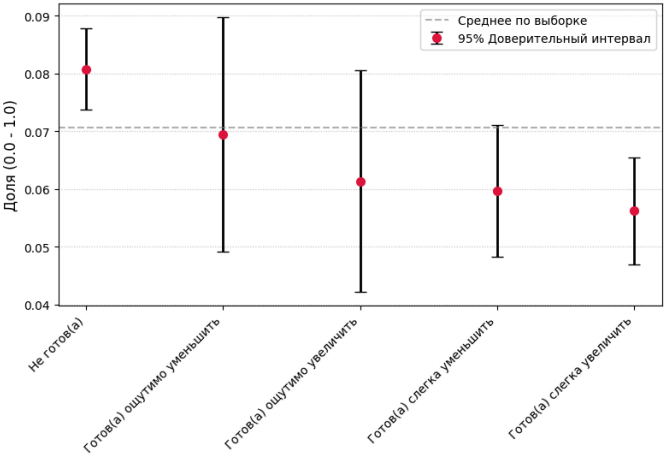
Распределение в разрезе признака: change_hours_numb



Анализ по категории: 'Средств хватает чтобы купить все, что нужно'
Порог Бонферрони (0.05 / 10): 0.00500
Найдено значимых различий: 1 из 10

Пара групп		p-value	Значимо (Бонферрони)
8	Готов(а) слегка увеличить vs Не готов(а)	0.00	True

Отличия групп "Средств хватает чтобы купить все, что нужно" по change_hours_numb



```
columns = [ 'employment', 'employment2', 'satisfaction', 'state_at_the_moment', 'shift_org_1', 'shift_reg_1',
            'shift_spec_1', 'shift_prof_1', 'gender', 'age', 'org_type', 'qualification', 'wage_rate', 'condition_org',
            'academic_degree', 'home_duty', 'change_hours_numb', 'expectation_prof',
            'schedule', 'avr_work_hours_grouped' ]

#отдельно печатать 'income_level',
#регион - 1 в моей выборке
#specialization - слишком много признаков
#числовые - age_number, 'Длина коммента', 'exper', 'exper_org', 'exper_reg', 'avr_work_hours', 'future_prestige', 'future_ec
# числовые про удовлетворенность 'future_justice', 'future_social_help', 'future_organisation', 'future_selfgrowth', 'futu
# числовые про удовлетворенность 'future_corpprocess', 'future_information', 'future_schedule', 'future_riskinformation',
# числовые про удовлетворенность 'future_region', 'future_region_healthcare'

#опросные фичи 'edu_1', 'edu_2', 'edu_3', 'edu_4', 'edu_5', 'edu_6', 'edu_7', 'edu_8', 'edu_9', 'edu_10', 'edu_11', 'edu_12
#аргументы 'burnout1_desc', 'burnout1_exh', 'burnout1_dist', 'burnout1_cog', 'burnout1_emt', 'burnout1_integral', 'burnout1_ir
```

for col in columns:			
get_differential_burnout_report(col)			
слегка ...			
6	Готов(а) ощутимо уменьшить vs Не готов(а)	0.00	True
8	Готов(а) слегка увеличить vs Не готов(а)	0.00	True
3	Готов(а) ощутимо увеличить vs Не готов(а)	0.00	True

Распределение в разрезе признака: employment

